КАЧЕСТВО И ТОЧНОСТЬ ПРОЕКТА

Полная защита по вопросам accuracy, precision, recall, FP/FN и инженерной надежности EH/Unwind Analyzer

Материал рассчитан на ситуацию, когда комиссия подробно спрашивает: как доказано качество, как оценить точность, что такое ложные срабатывания, почему нельзя назвать один процент и как построить строгую валидацию.

Главная позиция
Проект не является ML-классификатором вредоносности. Это rule-based анализатор PE x64 EH/Unwind metadata. Поэтому качество оценивается не одним числом, а системой метрик: parser correctness, finding-level precision/recall, benign false-positive rate, dynamic event recall, report validity, robustness, performance, portability and release readiness.

1. Почему нельзя отвечать одним процентом

Вопрос “какая точность?” выглядит простым, но для такого проекта он неполный. Точность зависит от того, какую именно задачу мы оцениваем. Analyzer может быть полностью корректен как parser, но иметь неполный набор эвристик для некоторых вмешательств. Он может не давать false positives на benign corpus, но это не доказывает recall на неизвестных obfuscation cases. Он может валидно формировать JSON, но это отдельное качество report contract, а не detection accuracy.

Поэтому правильный ответ начинается с уточнения: “точность чего именно?”. Для EH/Unwind Analyzer есть минимум четыре разных уровня оценки: file-level, record-level, finding-level и event-level. Если смешать их в одно число, получится красивая, но научно слабая цифра.

Уровень	Вопрос	Почему отдельный
File-level	Должен ли PE получить хотя бы один тревожный сигнал?	Просто для презентации, но грубо: файл с одной проблемой и файл с сотней проблем считаются одинаково.
Record-level	Правильно ли отмечена конкретная RUNTIME_FUNCTION или UNWIND_INFO?	Подходит для проверки parser-а и .pdata/.xdata logic.
Finding-level	Должен ли конкретный code из 27 появиться в отчете конкретного файла?	Лучший уровень для оценки 27 признаков: precision/recall можно считать per finding.
Event-level	Должно ли dynamic событие быть зафиксировано при выполнении?	Нужно для VEH/VCH/UEF/RtlAddFunctionTable/handler hit.

2. Что такое качество проекта

Качество продукта шире, чем detection accuracy. Для инструмента reverse engineering качество означает: он не падает на поврежденных файлах, правильно читает формат, объясняет вывод, сохраняет машинно-читаемый отчет, воспроизводимо работает через CLI, имеет тестовый corpus, умеет отделять жесткие structural violations от контекстных suspicion и честно показывает границы своих выводов.

Ось качества	Что проверяется	Как доказывать
Functional correctness	Функции проекта делают то, что заявлено: читают PE, .pdata, .xdata, формируют findings.	Unit tests, regression samples, expected findings.
Parser safety	Нет чтения за пределами, переполнений, crash на malformed input.	Bounds checks, mutation fuzz, partial reports.
Detection quality	Признаки срабатывают на правильных нарушениях и не срабатывают на нормальных случаях.	TP/FP/TN/FN на размеченном corpus.
Report quality	Отчет показывает evidence, matrix 27 features, key diagnostics, JSON schema.	unit_report_json/html, semantic validator, visual QA.
Dynamic validity	Runtime-события фиксируются в Windows dynamic mode.	Instrumented dynamic samples, event-level validation.
Usability	CLI понятен, exit codes предсказуемы, отчеты можно открыть и передать backend/frontend.	cli_contract, documentation, examples.
Maintainability	Код разделен на parser, heuristics, report, dynamic, tests.	Архитектура, CTest, release hygiene.
Release readiness	Поставка чистая, версии schema согласованы, лишние артефакты отсутствуют.	release_metadata, release_hygiene.

3. Ground truth

Ground truth - это эталонная разметка. Без нее нельзя честно считать TP, FP, TN, FN. В нашем проекте ground truth может быть создан разными способами: специально сконструированный PE, где мы точно знаем внесенное нарушение; ручная экспертная проверка; comparison с независимым parser-ом; сведения о compiler/linker provenance; instrumented dynamic target, который специально вызывает нужный handler или API.

Источник ground truth	Плюс	Минус
Synthetic sample	Точно известно, что именно сломано.	Не покрывает все разнообразие реальных toolchains.

Источник ground truth	Плюс	Минус
Expert manual review	Высокое доверие к сложным случаям.	Дорого, медленно, возможны расхождения экспертов.
Differential parsers	Можно сравнить с независимыми инструментами.	Инструменты тоже могут ошибаться.
Compiler provenance	Известно, чем и как собран benign sample.	Не всегда доступно для реальных PE.
Instrumented dynamic target	Можно точно ожидать VEH/UEF/RtlAddFunctionTable event.	Покрывает только контролируемый runtime path.

Фраза для комиссии
Нельзя говорить о false positive или false negative без ground truth. Если эталонной разметки нет, корректно говорить: “finding требует экспертной проверки”, а не “это точно ошибка инструмента” или “это точно атака”.

4. Confusion matrix

Confusion matrix - таблица четырех исходов сравнения результата analyzer-a с ground truth. Она применима не только в машинном обучении. Любой детектор, который выдает положительный/отрицательный сигнал, можно оценить через TP, FP, TN, FN.

Ground truth / результат	Analyzer дал сигнал	Analyzer не дал сигнал
Нарушение есть	TP: true positive, правильное обнаружение.	FN: false negative, нарушение пропущено.
Нарушения нет	FP: false positive, ложное срабатывание.	TN: true negative, правильное отсутствие сигнала.

В проекте “сигнал” лучше определять не как “файл вредоносный”, а как более узкое событие: конкретный finding code появился в отчете. Например, для finding-level оценки code rf_overlaps является сигналом, а ground truth говорит, есть ли реальное пересечение RUNTIME_FUNCTION ranges.

5. Метрики точности

Метрика	Формула	Смысл
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Доля правильных решений среди всех. Уязвима к дисбалансу классов.
Precision	TP / (TP + FP)	Когда инструмент сработал, насколько часто срабатывание подтверждается.
Recall / sensitivity	TP / (TP + FN)	Когда нарушение реально есть, насколько часто инструмент его находит.
Specificity	TN / (TN + FP)	Когда нарушения нет, насколько часто инструмент не создает ложный сигнал.
False Positive Rate	FP / (FP + TN)	Доля ложных тревог среди нормальных объектов.
False Negative Rate	FN / (FN + TP)	Доля пропусков среди объектов с реальным нарушением.
F1-score	2 * precision * recall / (precision + recall)	Баланс precision и recall; не учитывает TN.
NPV	TN / (TN + FN)	Если инструмент молчит, насколько часто это действительно нормальный случай.

Почему accuracy опасна: если в корпусе 990 benign и 10 anomalous samples, инструмент, который всегда отвечает “аномалий нет”, получит 99% accuracy, но recall по нарушениям будет 0%. Для triage-инструмента это неприемлемо. Поэтому на защите нужно говорить не только accuracy, а precision, recall, specificity и false-positive rate.

6. Score, confidence и accuracy - разные вещи

Понятие	Что это	Чем не является
Score	Внутренняя численная оценка подозрительности по найденным признакам.	Не процент вредоносности и не accuracy.
Confidence	Оценка силы evidence и надежности вывода в рамках эвристической модели.	Не ML probability.

Понятие	Что это	Чем не является
Verdict	Текстовая категория итогового вывода: low/moderate/high suspicion и т.п.	Не судебный приговор о вредоносности.
Accuracy	Внешняя экспериментальная метрика на размеченном corpus.	Не поле, которое analyzer может честно вывести сам без ground truth.
Finding	Технический сигнал о нарушении/аномалии.	Не утверждение “файл вредоносный”.

Если комиссия спросит, почему в отчете есть confidence, но ты говоришь, что accuracy не заявляется, ответ такой: confidence - это внутренняя эвристическая уверенность в evidence конкретного анализа. Accuracy - это внешнее статистическое свойство инструмента на benchmark. Это разные уровни.

7. Текущие evidence качества в проекте

В проекте есть несколько уже подготовленных доказательств качества. Они не равны глобальной accuracy, но являются сильными инженерными validation signals.

Evidence	Факт	Что доказывает	Что не доказывает
CTest	16/16 tests passed в последнем локальном прогоне.	Кодовые сценарии parser/heuristics/reports/CLI/fuzz/release проходят.	Не доказывает 100% detection accuracy на всех PE.
Extended corpus	78 PE: 60 static, 9 dynamic, 9 mixed.	Есть controlled samples для демонстрации и регрессии.	Не является большим реальным benchmark.
Reports	78 HTML, 78 JSON, 78 CSV, 78 Ghidra-CSV.	Выходные форматы формируются воспроизводимо.	Не доказывает экспертную истинность каждого вывода.
Matrix 27 features	HTML показывает все 27 признаков: active/inactive.	Покрытие отчетности по диагностической модели.	Не значит, что 27 признаков покрывают все возможные атаки.
Mutation fuzz	120 deterministic mutations внутри CTest.	Устойчивость к поврежденным входам.	Не является detection benchmark.
Release validation	В release-документах: 14/14 static labeled valid; 106/106 real benign valid; non-zero scores 0; 174795 runtime functions parsed.	Показывает отсутствие наблюдаемых FP-score на данном benign corpus и масштаб parsing validation.	Не переносится автоматически на все benign PE мира.

8. Как объяснять 16 тестов, 27 признаков и 78 бинарников

Это частый вопрос. 16 - количество CTest-сценариев. 27 - количество диагностических признаков. 78 - количество тестовых PE-входов в расширенном корпусе. Эти числа не обязаны совпадать, потому что один тестовый сценарий может проверять несколько признаков, а один PE может активировать несколько findings.

Число	Что означает	Роль в качестве
16	Автоматизированные CTest-сценарии.	Quality gate для кода и контрактов.
27	Эвристические признаки модели анализа.	Функциональная ширина диагностики.
78	Тестовые PE-бинарники.	Входной corpus для демонстрации и регрессии.
120	Mutation fuzz cases.	Robustness к поврежденным входам.
106	Benign real corpus из release-документов.	Проверка ложных тревог на конкретном benign наборе.

9. Как считать качество static-части

Класс проверки	Что проверяет	Метрика качества
PE layout correctness	raw/virtual OOB, overlaps, alignment, Exception Directory validity.	Precision/recall по structural findings; parser failure rate.
.pdata correctness	invalid ranges, function OOB, unsorted, overlaps, duplicate begin.	Record-level TP/FN по RUNTIME_FUNCTION entries.
.xdata correctness	UNWIND_INFO Version/Flags/CountOfCodes/UWOP slots/padding/tail.	Record-level parse accuracy и finding-level recall.
Placement/context	writable/executable metadata, handler nonexec/writable, unusual section.	Precision важна, потому что редкие benign layouts могут давать FP.

Класс проверки	Что проверяет	Метрика качества
Reuse/dominance	unwind reuse, handler dominance, combo rules.	Нужна benign calibration, чтобы central language handlers не давали лишние тревоги.
Report evidence	RVA, section, file offset, expected/available, failed_field.	Evidence completeness, schema validity, semantic consistency.

10. Как считать качество dynamic-части

Класс dynamic	Что проверяет	Метрика качества
Session reliability	ran, ok, timed_out, target_exited, instrumentation_complete.	Success rate dynamic sessions; timeout/failure rate.
Exception observation	exceptions_seen, first_chance_seen, second_chance_seen.	Event visibility на instrumented targets.
Handler observation	unwind_handler_hits, veh_hits, vch_hits, uef_hits.	Event-level recall: ожидаемый handler hit был зафиксирован или нет.
Runtime function tables	RtlAddFunctionTable, RtlInstallFunctionTableCallback.	Recall по instrumented dynamic function table samples.
Event evidence quality	events array: type/module/thread/address/args.	Полнота динамического отчета.
Trace quality	RIP trace после handler hit.	Наличие route evidence, но не полная реконструкция control flow.

11. False positives: что это и как снижать

False positive - ситуация, когда finding появился, но ground truth говорит, что нарушения нет. Для проекта это особенно важно, потому что инструмент предназначен для реверсеров: лишние тревоги создают ручную нагрузку.

Причина FP	Пример	Как снижать
Редкий compiler/toolchain layout	Metadata лежит не в привычной секции, но это допустимо.	Контекстные правила, severity, benign corpus calibration.
Protector/runtime без злого смысла	Central handler обслуживает много функций.	Handler dominance не считать автоматически атакой; показывать evidence.
JIT/runtime metadata	RtlAddFunctionTable используется легитимно.	Dynamic validation и manual interpretation.
Слишком жесткий threshold	Слабый сигнал стал сильным verdict.	Threshold tuning, weakening rules.
Неполный контекст отчета	Finding есть, но не показаны benign alternatives.	Добавлять explanation, next step, confidence.

В текущем проекте FP снижаются тем, что отчет не говорит “malware”, а показывает technical finding, severity, evidence, context и manual validation steps. Это корректная triage-позиция.

12. False negatives: что это и как снижать

False negative - ситуация, когда нарушение реально есть, но analyzer его не обнаружил. Для проекта это возможно, если вмешательство не нарушает текущие инварианты или находится вне зоны видимости static/dynamic слоя.

Причина FN	Пример	Как снижать
Согласованное изменение metadata	RUNTIME_FUNCTION и UNWIND_INFO изменены так, что формально выглядят корректно.	Добавить disassembly/prolog matching и semantic validation.
Нет проверки .text	UWOP заявляет одно, prolog делает другое.	Добавить Capstone/Zydis-like disassembler layer.
Runtime path не достигнут	Handler существует, но sample run не вызвал нужное исключение.	Targeted dynamic tests и input сценарии.
Ненаблюдаемый registration mechanism	Handler/function table регистрируется способом вне текущих breakpoints.	Расширить dynamic instrumentation.
Новый/редкий формат	Неподдержанная toolchain-specific metadata.	Расширить parser и corpus.

13. Performance, portability, security как качество

Комиссия может спросить не только про точность. Качество продукта также включает скорость, переносимость и безопасность использования.

Направление	Что измерять	Как отвечать
Performance	Время анализа файла, файлов/минуту, MB/s, память.	Сейчас основной упор на корректность и evidence; performance можно измерить batch-прогоном corpus.
Portability	Static на Windows/Linux/macOS, dynamic только Windows.	Static работает с PE как файлом на разных ОС; dynamic зависит от Windows Debug API.
Security	Не исполнять target в static mode; dynamic запускать в lab/sandbox.	Static безопаснее; dynamic требует контролируемой среды.
Reliability	Exit codes, partial reports, atomic writes.	cli_contract проверяет предсказуемое поведение.
Integration	JSON schema stability, CSV/Ghidra CSV.	JSON - контракт для backend/DB/frontend.

14. Как построить строгий benchmark

1. Определить задачу

Например: finding-level detection для каждого из 27 codes, а не “вредоносный/не вредоносный”.

2. Определить positive/negative classes

Для rf_overlaps positive - реальные пересечения ranges; negative - корректные непересекающиеся ranges.

3. Собрать corpus

Synthetic positives, benign negatives, real-world PE, compiler-known samples, protector/JIT samples.

4. Разметить ground truth

Записать, что именно сломано, где RVA, кто проверил, чем подтверждено.

5. Зафиксировать команды запуска

Одна версия analyzer, одинаковые флаги, сохранение JSON outputs.

6. Посчитать confusion matrix

TP/FP/TN/FN отдельно для каждого finding code и каждого уровня оценки.

7. Посчитать metrics

Precision, recall, specificity, FPR, FNR, F1; отдельно для static и dynamic.

8. Разобрать ошибки

Каждый FP и FN превращается в issue: правило, threshold, parser, corpus, документация.

9. Повторить после правок

Regression testing не должен ломать уже закрытые классы.

15. Пример расчета метрик

Допустим, мы оцениваем finding rf_overlaps на finding-level. Есть 40 PE с настоящим overlap и 100 PE без overlap. Analyzer нашел overlap в 38 positive PE и ошибочно нашел overlap в 2 negative PE. Тогда: TP=38, FN=2, FP=2, TN=98.

```
precision = 38 / (38 + 2) = 95%
recall = 38 / (38 + 2) = 95%
specificity = 98 / (98 + 2) = 98%
accuracy = (38 + 98) / 140 = 97.14%
```

Но если для другого finding, например handler_dominance, precision ниже из-за легитимных central handlers, общий средний процент может скрыть эту проблему. Поэтому метрики надо считать per finding class.

16. Как отвечать на жесткие вопросы

Как вы оцениваете качество проекта?

Многоуровнево: CTest, regression samples, extended PE corpus, schema/semantic validation, mutation fuzz, benign corpus checks, dynamic validation и качество evidence в отчетах.

Какая точность?

Глобальную accuracy не заявляю без большого размеченного benchmark. Для текущей версии есть инженерная валидация; строгая точность должна считаться через TP/FP/TN/FN per finding class.

Почему нет одного процента?

Потому что parsing accuracy, finding recall, benign specificity, dynamic event recall и report validity - разные свойства. Один процент смешал бы разные задачи.

Можно ли сказать, что false positives отсутствуют?

Только на конкретном проверенном benign corpus при данном threshold. Нельзя утверждать отсутствие FP во всей мировой совокупности PE.

Можно ли сказать, что false negatives отсутствуют?

Нет. FN возможны, если нарушение не попадает в текущие инварианты или runtime path не достигнут. Это честное направление развития.

Что сильнее всего доказывает качество?

Traceability: требование -> test sample -> expected finding -> actual JSON/HTML -> automated validation -> readable evidence.

Почему это не ML accuracy?

Потому что нет обучаемой модели. Признаки rule-based. ML-метрики используются только для внешней оценки детектора на размеченном corpus.

Что бы вы улучшили для измерения качества?

Собрал бы большой per-finding benchmark, добавил record-level разметку, disassembly/prolog matching, Windows dynamic CI и расчет precision/recall по каждому классу.

17. Готовый ответ на 2 минуты

“Качество проекта я оцениваю не одним процентом, потому что это не нейросетевой классификатор вредоносности, а инженерный анализатор PE x64 EH/Unwind metadata. У него несколько разных уровней качества: корректность парсера, корректность findings, отсутствие ложных тревог на benign corpus, устойчивость к поврежденным PE, валидность отчетов, dynamic validation и релизная готовность. Последний локальный gate прошел 16/16 CTest; расширенный корпус содержит 78 PE и по ним сгенерированы HTML/JSON/CSV/Ghidra-CSV; в release-документах также зафиксированы проверки labeled и benign corpus.”

“Если говорить строго о точности, ее надо считать на размеченном benchmark. Для каждого finding code из 27 нужно определить ground truth и посчитать TP, FP, TN, FN. Затем отдельно считать precision, recall, specificity, false-positive rate и false-negative rate. Я сознательно не называю необоснованную 100% accuracy, потому что это было бы некорректно без большого репрезентативного corpus. Вместо этого

проект показывает воспроизводимую инженерную валидацию и готовый план строгой оценки качества.”

18. Короткая шпаргалка

Если спрашивают	Ответ в одну фразу
Качество?	Многоуровневая инженерная валидация: parser, heuristics, reports, CLI, fuzz, corpus, dynamic.
Точность?	Строго считается через TP/FP/TN/FN на размеченном benchmark; глобальную accuracy без benchmark не заявляю.
False positives?	Возможны; снижаются context rules, severity, confidence, benign validation и manual review.
False negatives?	Возможны; снижаются расширением правил, disassembly/prolog matching и targeted dynamic.
Почему не ML?	Форматные инварианты лучше проверяются rule-based и объяснимо; ML можно добавить только как triage layer.
Что доказано?	Текущий quality gate и corpus validation; не абсолютная мировая точность.

19. Итог

Сильная позиция на защите: проект имеет проверяемое инженерное качество, но его точность надо описывать строго. Не обещать невозможное, а показывать evidence. Не говорить “100% accuracy”, а говорить “16/16 CTest passed, 78 PE corpus processed, reports validated, benign corpus checks, mutation fuzz, and a defined methodology for TP/FP/TN/FN measurement”. Это звучит профессионально и защищает от попытки комиссии поймать на некорректной статистике.

ПРИЛОЖЕНИЕ: FINDINGS, SCORE И ОБОСНОВАНИЕ ВЕСОВ

Что важно сказать комиссии
Число score в отчете - это не процент вредоносности, не ML probability и не статистическая accuracy. Это инженерная шкала 0..100, которая суммирует силу обнаруженных технических признаков. Чем сильнее нарушение формата и чем больше независимых семейств evidence, тем выше score.

20. Как устроен score

В коде проекта score хранится как целое число от 0 до 100. Каждое finding добавляет фиксированный или условный вес. После сложения значение ограничивается диапазоном 0..100. Затем suspicion вычисляется как score / 100. Это удобно для отчета, но это не означает вероятность атаки. Например, score 25 означает “накоплено 25 баллов статической подозрительности”, а не “25% файл вредоносный”.

Поле отчета	Как считается	Как объяснять
score	Сумма весов сработавших признаков, clamp 0..100.	Инженерная оценка силы статических сигналов.
suspicion	score / 100.	Нормированное представление score, а не вероятность.
verdict	>=70 high, >=40 moderate, >=15 low, иначе no strong anomalies.	Категория для triage, а не окончательный диагноз.
evidence_level	strong/medium/weak по числу high/medium findings и score.	Сила evidence, а не accuracy.
confidence	База + бонусы за семьи/сильные признаки - penalties.	Насколько надежно выглядит вывод в рамках правил.

21. Почему веса именно такие

Веса сгруппированы по смыслу evidence. Самые большие веса получают hard structural/integrity violations: они ломают базовую карту PE/.pdata/.xdata и могут сделать unwind ненадежным. Средние веса получают placement anomalies: они подозрительны, но могут иметь benign alternatives. Небольшие веса получают context/combo-признаки или одиночные слабые сигналы: они усиливают картину, но редко должны быть единственным основанием для сильного вывода.

Группа	Типичный вес	Почему
Жесткие нарушения PE/.pdata/.xdata	20-25	Ломают маппинг, диапазоны функций или чтение UNWIND_INFO; это сильное техническое evidence.
Синтаксические anomalies UNWIND_INFO	10-25	Зависят от распространенности: единичный сбой слабее массового сбоя.
Placement anomalies	8-20	Подозрительны, но возможны редкие benign layout/runtime cases.
Reuse/dominance	10-20	Может быть нормальным runtime/language pattern, поэтому требует контекста.
Combo-признаки	8-10	Не заменяют базовые findings, а усиливают связку нескольких сигналов.

22. Пороги процентов и ratios

В scoring есть не только фиксированные веса, но и пороги распространенности. Это сделано, чтобы отличать одиночную локальную проблему от системного нарушения по множеству записей.

Порог	Где используется	Смысл
>1% entries	invalid_ranges, rf_oob, unwind_rva_oob, unwind_parse_fail.	Если нарушение затрагивает больше 1% .pdata, оно считается не случайной единичной проблемой, а более широкой деградацией карты.
>5% entries	unwind_anomalies, unusual/writable placement high weight.	Массовость anomalies/placement делает сигнал сильнее.
>10% reuse и n > 200	unwind_reuse_moderate.	Умеренный reuse значим только на достаточно большой таблице и при наличии другого сигнала.
>30% reuse и n > 200	unwind_reuse_strong.	Один UNWIND_INFO обслуживает слишком большую долю функций; это сильный pattern.

Порог	Где используется	Смысл
handler_count > 50 и reuse > 80%	handler_dominance.	Один handler доминирует над большим числом функций; без контекста это suspicious dispatcher-like pattern.

23. Полная таблица weights по 27 findings

Finding code	Вес	Severity	Почему такая оценка
pe_section_raw_oob	25	high	Raw-диапазон секции выходит за файл. Это ломает доверие к чтению байтов.
pe_section_virtual_oob	20	high	Virtual-диапазон секции выходит за SizeOfImage. Это ломает карту RVA image.
pe_section_raw_overlap	15	medium	Raw-диапазоны секций пересекаются. Сигнал серьезный, но raw overlap может быть менее критичен, чем virtual ambiguity.
pe_section_virtual_overlap	20	high	RVA-диапазоны пересекаются. Один RVA может соответствовать нескольким секциям, поэтому это сильнее raw overlap.
pe_section_misaligned	8	medium	Alignment/header invariant нарушен. Вес ниже, потому что часть нестандартных файлов может иметь странный, но не атакующий layout.
pe_exception_directory_oob	25	high	Exception Directory не читается корректно. Для x64 unwind это вход в .pdata, поэтому вес максимальный.
no_runtime_functions	10	medium	В x64 image с executable code отсутствие .pdata может быть допустимо для leaf-only/simple image, поэтому вес умеренный.
invalid_ranges	10/20	medium/high	BeginAddress >= EndAddress. Если >1% записей, вес 20; единичное нарушение весит 10.
rf_oob	10/25	medium/high	Function range выходит за SizeOfImage. Массовость >1% повышает вес до 25.
unwind_rva_oob	10/25	medium/high	UnwindInfoAddress не ведет к допустимым байтам. Массовость >1% повышает вес до 25.
rf_unsorted	15	medium	Таблица не отсортирована. Это мешает однозначному поиску, но не всегда означает corruption metadata.
rf_overlaps	25	high	Диапазоны функций пересекаются. Один ControlPc может получить неоднозначную unwind-запись.
duplicate_begin	10	medium	Повтор BeginAddress подозрителен, но может требовать контекста, поэтому вес умеренный.
unwind_parse_fail	10/25	medium/high	UNWIND_INFO не разбирается. Если >1% записей, вес 25, потому что metadata системно ненадежна.
unwind_anomalies	10/20	medium/high	UNWIND_INFO читается, но поля/операции странные. Если >5% записей, вес повышается.
unwind_reuse_strong	20	high	>30% функций при n>200 используют один UnwindInfoRVA. Это сильный pattern централизованной metadata.
unwind_reuse_moderate	10	medium	>10% reuse при n>200, но только если есть другой static signal. Сам по себе умеренный reuse не пугает.
handler_dominance	15	medium	Один handler обслуживает >80% handlers при handler_count>50. Вес средний из-за возможных language/runtime handlers.
unwind_in_exec_section	20	high	Metadata лежит в executable section. Это смешивает код и metadata, что подозрительно.
unwind_unusual_section	8/15	medium/high	UNWIND_INFO в нетипичной секции. Если >5% записей, вес 15; иначе 8.
unwind_writable_section	10/18	medium/high	Metadata находится в writable section. Массовость >5% повышает вес до 18.
handler_nonexec_section	15	high	Handler RVA ведет в non-executable/unknown область. Handler должен быть кодом.
handler_writable_section	10	medium	Handler target writable. Подозрительно, но может требовать dynamic/manual проверки.
placement_exec_nonexec_handler_combo	10	high	Executable metadata + non-executable handler вместе сложнее объяснить benign layout.
placement_integrity_combo	8	medium	Suspicious placement подтверждается integrity-проблемами. Это усилитель, а не базовая причина.
reuse_dominance_combo	8	medium	Unwind reuse + handler dominance похожи на dispatcher-style pattern.
writable_metadata_handler_combo	10	high	Writable metadata + writable handler target вместе являются сильной необычной связкой.

24. Confidence: почему может снижаться

Score показывает силу сигналов, но confidence показывает, насколько надежен сам вывод. Confidence начинается с базы 0.15. При нулевом score confidence ставится 0.20 как “нет признаков, но это не доказательство полной чистоты”. Затем confidence растет за разные evidence families, high/medium findings и hard table/unwind problems. Но confidence снижается, если сигнал одиночный, sample маленький, evidence только из placement или только из одного семейства.

Элемент confidence	Значение	Смысл
Base	+0.15	Минимальная стартовая уверенность при наличии анализа.
Zero score base	0.20	Если score=0, это аккуратная уверенность в отсутствии сильных сигналов, не абсолютная чистота.
Per evidence family	+0.11	Разные семейства evidence повышают доверие: table, unwind, placement, reuse, dominance.
Per strong finding	+0.08	High/critical findings усиливают вывод.
Per medium finding	+0.04	Medium findings добавляют умеренную уверенность.
Hard table bonus	+0.08	Invalid ranges/OOB/overlaps - жесткие проблемы карты .pdata.
Hard unwind bonus	+0.08	Parse failures/anomalies - жесткие проблемы .xdata.
Single signal penalty	-0.12	Один несильный сигнал не должен давать чрезмерную уверенность.
Small sample penalty	-0.06	Малая .pdata может завышать ratios.
Placement-only penalty	-0.10	Placement-only может быть benign uncommon layout.
Single-family penalty	-0.08	Один тип evidence слабее cross-family correlation.
Bounds	0.05..0.90	Confidence никогда не становится 0 или 1.

25. Примеры объяснения score

Пример 1. Если sample 77 имеет unwind_parse_fail из-за CountOfCodes/UWOP operand slots и это затрагивает 2 из 3 записей, доля больше 1%, поэтому finding получает high branch: +25. Итоговый score может быть 25, verdict low_suspicion, потому что это один основной класс проблемы: серьезная ошибка metadata есть, но нет множества независимых семейств evidence.

Пример 2. Если один invalid_range найден в таблице из 300 записей, это 0.33%, то применяется low branch +10. Если invalid ranges затрагивают больше 1%, применяется high branch +20. Логика такая: единичная поврежденная запись слабее массовой деградации таблицы.

Пример 3. Writable metadata (+10 или +18) и writable handler (+10) вместе дополнительно дают writable_metadata_handler_combo (+10). Комбинация важна, потому что одна writable metadata может быть редкостью toolchain-a, но writable metadata плюс writable handler target выглядит значительно более подозрительно.

Короткий ответ комиссии
Веса назначены не случайно. Они отражают инженерную силу evidence: hard structural/unwind violations получают высокий вес, контекстные placement/reuse признаки - средний, combo-признаки - дополнительный усилитель. Пороги 1%, 5%, 10%, 30% и 80% нужны, чтобы отличать одиночные аномалии от системного паттерна.